Plan de Desarrollo: GymSasS (MVP)

Documento de Planificación Estratégica y Asignación de Tareas • ISFT 194
Objetivo: Entrega Funcional, Defendible y Aprobación del TP Final (Noviembre 2026)

Este documento establece el cronograma de trabajo estructurado para el desarrollo del Producto Mínimo Viable (MVP) del sistema **GymSasS**. El enfoque principal estará en cumplir estrictamente con los criterios de evaluación de la cátedra: arquitectura cliente-servidor, separación de responsabilidades (Controllers/Services), uso exhaustivo de Pydantic y SQLAlchemy, y una correcta integración Frontend-Backend.

Equipo y Distribución de Roles

Líder Chelo

Tech Lead / PM

- Diseño y modelado de Base de Datos.
- Arquitectura base (FastAPI).
- Lógica de negocio compleja (Service).
- Autenticación (JWT/Hash).
- Code Reviews y dirección del equipo.

Front Eros

Frontend Developer

- Maquetado completo (HTML/CSS/JS).
- Consumo de API REST (Fetch/Axios).
- Manejo de estado en vistas.
- Captura y renderizado de errores 4xx.
- UX enfocada en defender el sistema.

Back Hugo

Backend / Fullstack Jr.

- Endpoints CRUD (Controllers).
- Schemas Pydantic (Validaciones).
- Operaciones SQLAlchemy (Models).
- Apoyo en integración HTTP.
- Documentación Swagger.

Cronograma General

Hito de Entrega: Finales de Noviembre de 2026. Quedan aproximadamente 6 semanas de desarrollo activo más 1 semana de estabilización ("Code Freeze") y preparación de la defensa técnica.



Fase 1: Fundación, Arquitectura y Accesos

Semana 1: Setup y Base de Datos

Mediados de Octubre

Objetivo: Dejar lista la estructura del proyecto en GitHub y levantar los modelos de SQLAlchemy. Sin base sólida no hay sistema.

Chelo Tech Lead	**Arquitectura y DB:** Crear repositorio Git con `.gitignore`. Definir estructura de carpetas (`main.py`, `controllers`, `services`, `models`). Crear conexión a BD relacional y diseñar modelos SQLAlchemy para el MVP (Usuario, Pago, Cuota).
Eros Frontend	**Boilerplate y Vistas base:** Crear estructura HTML/CSS base. Maquetar la pantalla exclusiva de "Bienvenida y Registro de Clientes" (RF-02).
Hugo Backend	**Schemas Pydantic:** Investigar estructura del repo. Crear `UsuarioCreate`, `UsuarioResponse`, garantizando tipos y campos obligatorios (Nombre, Apellido, DNI, Mail, Teléfono) (RF-01).

Semana 2: Autenticación, Seguridad y CRUD

Fines de Octubre

Objetivo: Un usuario debe poder registrarse, el admin debe poder habilitarlo, y el sistema debe autenticar con JWT.

Chelo	**Autenticación y Guards:** Implementar endpoint `POST /auth/login`. Hashear contraseñas (bcrypt) y emitir JWT. Configurar dependencias en FastAPI para proteger rutas (solo admin o usuario logueado).
Eros	**Integración HTTP (Auth):** Conectar el form de Login y Registro con la API. Guardar el token en `localStorage` o `sessionStorage`. Capturar el código 401 `Unauthorized` y mostrar un mensaje visual amigable.
Hugo	**CRUD de Usuarios:** Desarrollar endpoints `GET /usuarios`, `POST /usuarios` y `PATCH /usuarios/{id}/habilitar` (RF-03). Separar estrictamente la validación HTTP (Controller) de la escritura en base de datos (Service).

Fase 2: Reglas de Negocio, Validaciones y Robustez

Semana 3: Pagos y Vencimientos (Core MVP)

Principios de Noviembre

Objetivo: Implementar la funcionalidad más crítica pedida por el cliente: asentar cobros y calcular fechas de caducidad.

Chelo Tech Lead	**Lógica de Negocio (Services):** Desarrollar la lógica para asentar un pago (Efectivo/MP/Transferencia). Al registrar un pago válido, el sistema debe actualizar o generar automáticamente la fecha de vencimiento de la cuota (RF-08, RF-12).
Eros Frontend	**Panel Administrativo:** Maquetar el Dashboard del Administrador. Crear vista para listar usuarios y el formulario/modal para registrar el pago de un usuario específico, enviando el token en los headers.
Hugo Backend	**Historial de Pagos:** Desarrollar endpoint `GET /pagos/{usuario_id}`. Traer historial ordenado por fecha (RF-09). Asegurarse de retornar un esquema Pydantic limpio sin exponer datos sensibles.

Semana 4: Manejo de Errores y Notificaciones

Mediados de Noviembre

Objetivo: Evitar que la app se rompa ante malos ingresos de datos y notificar al cliente sus vencimientos próximos.

Chelo	**Sistema de Alertas:** Crear un endpoint que exponga qué cuotas vencen en los próximos X días (RF-13). Hacer *Code Review* del código de Hugo y Eros para asegurar cumplimiento de las pautas del profesor (Códigos HTTP correctos: 200, 201, 400, 404, 422).
Eros	**Feedback Visual y "Notificaciones":** Al consumir el endpoint de vencimientos de Chelo, renderizar un banner de advertencia para el cliente (RF-10). Atrapar códigos de error de la API (ej: Mail ya registrado) y mostrar alertas en pantalla (no en consola).
Hugo	**Robustez Pydantic:** Agregar validaciones estrictas: longitud de DNI, formato de email, y evitar registrar emails o DNIs duplicados (chequeo en Service, retorno 409 Conflict).

Nota para el Code Review: Durante esta semana, vos (Chelo) deberás revisar que ningún Controller tenga consultas SQLAlchemy directamente en su cuerpo. Todo llamado a DB debe pasar por los archivos dentro de la carpeta `services/`.

Fase 3: Integración E2E, Documentación y Defensa

Semana 5: Integración Total y Documentación

Tercera Semana de Noviembre

- **Chelo:** Redactar el archivo `README.md`. Debe incluir tecnologías, pasos de instalación (virtualenv, pip install), configuración de la base de datos (variables de entorno) y comandos de ejecución (uvicorn). Verificar que Swagger (`/docs`) muestre todo correcto.
- **Eros & Hugo:** Pruebas End-to-End (E2E). Deberán sentarse juntos y ejecutar el flujo completo en el navegador: Registrar Cliente → Admin Inicia Sesión → Admin Habilita Cliente → Admin Registra Pago → Cliente Inicia Sesión y ve su pago/vencimiento.

Semana 6: Code Freeze y Preparación de Defensa

Cuarta Semana de Noviembre

- **Equipo Completo:** *Code Freeze* (no se desarrollan funcionalidades nuevas, solo se arreglan bugs).
- **Chelo (Rol de Profesor):** Deberás hacer simulacros de examen con Eros y Hugo. Preguntarles: *¿Cómo viaja la petición HTTP desde el fetch hasta la Base de Datos? ¿Qué hace Pydantic acá? ¿Por qué usamos este código 422?*
- **Eros & Hugo:** Estudiar la estructura del proyecto. El requisito del TP exige que **todos** los integrantes puedan explicar el código, caso contrario afectará la nota grupal.

Semana 7: Entrega Final

Fines de Noviembre

Clonar el repositorio en una carpeta limpia de otra computadora y verificar que el sistema "levanta" correctamente siguiendo exclusivamente las instrucciones del `README.md`. Generar el archivo `requirements.txt` actualizado y hacer el push final a la rama principal (main).

Checklist Final del Docente (Aprobación Segura)

- [ ] ¿El proyecto se levanta desde cero siguiendo el README?
- [ ] ¿Swagger muestra correctamente los modelos y la API?
- [ ] ¿El frontend consume endpoints reales y no tiene datos hardcodeados?
- [ ] ¿El login funciona y las contraseñas están hasheadas?
- [ ] ¿Existe separación estricta entre Controller, Service y Models?
- [ ] ¿Todos (Chelo, Eros y Hugo) pueden demostrar un flujo completo y explicarlo?